

Методы оптимизации — подробный конспект к зачёту

27 апреля 2026 г.

Для кого этот текст и как им пользоваться

Документ рассчитан на **первое изучение**: к каждому методу добавлены пояснения “с нуля”, численные идеи и формулы, которые обычно ожидают на зачёте. В конце разделов иногда даются ссылки на Wikipedia как дополнительное чтение.

Каждый метод ниже можно читать по шаблону:

1. **Зачем** — какую задачу решаем и что на выходе.
2. **Что нужно заранее** — какие данные и предположения (гладкость, унимодальность и т.д.).
3. **Идея простыми словами** — без формул или почти без них.
4. **Шаги алгоритма** — что делать на одной итерации.
5. **Формулы** — то, что обычно спрашивают на зачёте.
6. **Остановка и практика** — когда заканчивать и на что смотреть в реализации.

Если что-то кажется абстрактным, прочитайте сначала только пункты 1–3, затем вернитесь к формулам.

Содержание

1	Постановка задач оптимизации	3
1.1	Что такое оптимизация в одном абзаце	3
1.2	Безусловная оптимизация	3
1.3	Условная оптимизация	3
1.4	Локальный и глобальный минимум	3
2	Теория: условия оптимальности и выпуклость	3
2.1	Градиент и матрица Гессе (напоминание)	3
2.2	Необходимые условия экстремума (без ограничений)	4
2.3	Достаточные условия (без ограничений)	4
2.4	Выпуклая оптимизация	4
2.5	Условия ККТ (Каруша–Куна–Таккера)	4

3	Одномерная оптимизация	5
3.1	Унимодальность	5
3.2	Метод перебора (сетка)	5
3.3	Метод деления пополам (дихотомия)	5
3.4	Метод золотого сечения	5
3.5	Метод Фибоначчи	6
3.6	Метод Свенна (расширение интервала)	6
3.7	Параболическая интерполяция (квадратичная аппроксимация)	6
4	Многомерная безусловная оптимизация: методы спуска в $\mathbb{R}^n$	7
4.1	Направление спуска	7
4.2	Линейный поиск (подбор шага α)	7
4.3	Градиентный спуск	7
4.4	Метод наискорейшего спуска	8
4.5	Метод сопряжённых градиентов (для квадратичной функции)	8
4.6	Метод Ньютона (подробно для начинающих)	9
4.6.1	Одна переменная: от ряда Тейлора до формулы “минус первая вторая”	9
4.6.2	Много переменных: то же самое, но “вторая производная” — матрица	10
4.6.3	Как это считают на практике (и почему не “инвертируют Гессиан”)	10
4.6.4	Когда шаг Ньютона “хороший”, а когда опасный	10
4.7	Модификации Ньютона (что именно улучшают)	11
4.8	Квазиньютоновские методы (DFP, BFGS) — полная запись	11
4.8.1	Каркас алгоритма квазиньютоновского спуска	11
4.8.2	DFP (обновление аппроксимации Гессиана B_k)	11
4.8.3	BFGS (обновление B_k)	11
4.8.4	BFGS (обновление $H_k \approx B_k^{-1}$, удобно в реализации)	12
4.8.5	Сложность и L-BFGS	12
5	Без производных	12
5.1	Метод Гаусса–Зейделя (покоординатный спуск) — как читать формулу	12
5.2	Метод Хука–Дживса (pattern search)	12
5.3	Метод Пауэлла (метод сопряжённых направлений без градиента)	13
5.4	Нелдер–Мид: симплекс в $\mathbb{R}^n$	14
6	Штрафы и барьеры	15
6.1	Метод штрафных функций	15
6.2	Метод барьерных функций (внутренние точки)	15
7	Линейное программирование (ЛП)	16
8	Глобальная оптимизация	16
8.1	Рой частиц (PSO, Particle Swarm Optimization)	16
8.2	Генетический алгоритм (ГА)	17
9	Имитация отжига	17
9.1	Пороговый алгоритм (мотивация)	17
9.2	Правило Метрополиса (стандарт SA для минимизации)	18
9.3	Полный псевдокод (с лучшим найденным решением)	18
9.4	Законы охлаждения (как в методичке)	18
9.5	Практические советы	18

1 Постановка задач оптимизации

1.1 Что такое оптимизация в одном абзаце

Нужно выбрать *решение* x (набор чисел: параметры модели, режим работы, маршрут и т.д.) так, чтобы некоторая *целевая функция* $f(x)$ была как можно меньше (или больше). Множество допустимых x может быть ограничено равенствами и неравенствами — тогда говорят об **условной** оптимизации.

1.2 Безусловная оптимизация

Задача: найти точку (или набор точек), где f минимальна на всём $\mathbb{R}^n$:

$$x^* \in \arg \min_{x \in \mathbb{R}^n} f(x).$$

Важно: минимум может не достигаться (например, $f(x) = e^{-x}$ на $\mathbb{R}$). Тогда ищут $\inf f$ и последовательность x_k , для которой $f(x_k)$ стремится к нижней грани.

1.3 Условная оптимизация

Задача:

$$x^* \in \arg \min_{x \in \Omega} f(x), \quad \Omega = \{x : g_i(x) \leq 0, i = 1, \dots, m; h_j(x) = 0, j = 1, \dots, p\}.$$

Смысл: часть решений “отбрасывается” ограничениями; оптимум может лежать *внутри* области или *на границе* (тогда активны некоторые $g_i = 0$).

1.4 Локальный и глобальный минимум

- **Глобальный минимум** x^* : $f(x^*) \leq f(x)$ для *всех* допустимых x . Это “самая низкая точка на всей карте рельефа”.
- **Локальный минимум:** существует “окрестность” (маленький шар вокруг точки), внутри которой x^* лучше всех. Вне окрестности могут быть точки с меньшим значением f .

Почему это важно: большинство классических численных методов (градиентный спуск, Ньютон, BFGS) в *лучшем случае* находят локальный минимум или стационарную точку. Чтобы искать глобальный минимум, подключают мултистарт, штрафы к глобальным эвристикам (PSO, ГА) или имитацию отжига.

2 Теория: условия оптимальности и выпуклость

2.1 Градиент и матрица Гессе (напоминание)

Если f гладкая, то **градиент** $\nabla f(x)$ — это вектор частных производных: он указывает направление *наибольшего* роста f в окрестности точки. **Гессиан** $H(x) = \nabla^2 f(x)$ — матрица вторых производных; он описывает “кривизну” функции в разных направлениях.

2.2 Необходимые условия экстремума (без ограничений)

Если x^* — локальный минимум и f дифференцируема в x^* , то

$$\nabla f(x^*) = 0.$$

Интуиция: в минимуме “наклон” в любом направлении не может быть отрицательным и положительным одновременно; значит линейная часть приращения f обнуляется.

Если f дважды дифференцируема, то дополнительно $H(x^*)$ **положительно полуопределена**: $H(x^*) \succeq 0$ (все собственные значения ≥ 0).

2.3 Достаточные условия (без ограничений)

Если $\nabla f(x^*) = 0$ и $H(x^*) \succ 0$ (все собственные значения > 0), то x^* — **строгий** локальный минимум. Интуитивно: функция ведёт себя как “вогнутая параболоид вверх” около точки.

2.4 Выпуклая оптимизация

Функция f **выпукла** на выпуклом множестве X , если отрезок между любыми двумя точками графика лежит не ниже хорды:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y), \quad \lambda \in [0, 1].$$

Для дифференцируемой выпуклой функции верно неравенство:

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle.$$

Главный вывод для зачёта: если f выпукла на X , то любая точка с $\nabla f = 0$ — **глобальный** минимум на X . Это сильно упрощает анализ.

2.5 Условия ККТ (Каруша–Куна–Таккера)

Рассмотрим

$$\min f(x) \quad \text{при} \quad g_i(x) \leq 0, \quad h_j(x) = 0.$$

Лагранжиан:

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \nu_j h_j(x).$$

При стандартных условиях регулярности (например, LICQ: градиенты активных ограничений линейно независимы) в локальном минимуме x^* существуют множители λ^*, ν^* такие, что

$$\begin{aligned} \nabla_x L(x^*, \lambda^*, \nu^*) &= 0, \\ g_i(x^*) &\leq 0, \quad h_j(x^*) = 0, \\ \lambda_i^* &\geq 0, \\ \lambda_i^* g_i(x^*) &= 0 \quad (\text{дополняющая нежёсткость}). \end{aligned}$$

Смысл дополняющей нежёсткости: если ограничение не активно ($g_i(x^*) < 0$), то $\lambda_i^* = 0$; если активно ($g_i = 0$), множитель λ_i^* может быть положительным — он “подталкивает” к допустимости.

3 Одномерная оптимизация: поиск минимума на отрезке

Постановка: $\min_{x \in [a, b]} f(x)$.

3.1 Унимодальность

Функция f **унимодальна** на $[a, b]$, если существует точка $x^* \in [a, b]$ такая, что f строго убывает слева от x^* и строго возрастает справа (в широком смысле — не возрастает/не убывает). Тогда на $[a, b]$ один “главный” минимум; методы сужения отрезка опираются именно на это свойство.

3.2 Метод перебора (сетка)

Зачем: самый простой способ получить приближённый минимум, если не хочется думать про свойства f .

Идея: разбить $[a, b]$ на равные отрезки шага h , вычислить f в узлах, взять узел с наименьшим значением.

Формулы: узлы $x_j = a + jh$, $j = 0, 1, \dots, N$, где $N \approx (b - a)/h$.

Оценка: $\Theta(1/h)$ вычислений f . Точность по аргументу порядка h .

Минусы: при малом h очень дорого; при большом h можно “промахнуться” мимо узкого минимума.

3.3 Метод деления пополам (дихотомия)

Зачем: сузить отрезок, содержащий минимум унимодальной функции, за логаритмическое число шагов.

Идея: возьмем две точки $x_1 < x_2$ внутри текущего отрезка $[a_k, b_k]$. Сравним $f(x_1)$ и $f(x_2)$. Унимодальность гарантирует: минимум *не может* оказаться сразу с обеих сторон “плохой” половины — отрезок можно сократить.

Шаги:

1. Положить $x_1 = \frac{a_k + b_k}{2} - \delta$, $x_2 = \frac{a_k + b_k}{2} + \delta$ с малым $\delta > 0$.
2. Если $f(x_1) \leq f(x_2)$, то новый отрезок $[a_k, x_2]$; иначе $[x_1, b_k]$.
3. Повторять, пока длина отрезка $> \varepsilon$.

Про δ : без него $x_1 = x_2$ и сравнение бессмысленно; δ берут много меньше требуемой точности по x .

3.4 Метод золотого сечения

Зачем: как дихотомия, но на каждом шаге нужно только **одно новое** вычисление f (вторая точка “наследуется” с прошлого шага).

Идея: разместить две внутренние точки симметрично относительно концов отрезка так, чтобы при отбрасывании части отрезка одна из точек осталась внутри нового отрезка в том же “золотом” отношении.

Формулы: $\tau = \frac{\sqrt{5}-1}{2} \approx 0,618$. Для $[a, b]$:

$$x_1 = b - \tau(b - a), \quad x_2 = a + \tau(b - a).$$

Если $f(x_1) \leq f(x_2)$, новый отрезок $[a, x_2]$; иначе $[x_1, b]$. Длина отрезка уменьшается примерно в τ раз за итерацию.

Сколько итераций до точности ε . Если начальная длина $L_0 = b - a$, то после k шагов $L_k \approx \tau^k L_0$. Отсюда $k \gtrsim \frac{\ln(L_0/\varepsilon)}{\ln(1/\tau)}$.

Почему “золотое”. Отношение τ — единственное положительное число, для которого при симметричном размещении двух точек внутри отрезка одна из них автоматически попадает внутрь нового отрезка с тем же относительным положением — поэтому на шаге нужна только одна новая оценка f .

3.5 Метод Фибоначчи

Отличие от золотого сечения. Если заранее зафиксировано ровно N вычислений f , метод Фибоначчи подбирает положения точек так, чтобы **минимизировать финальную длину интервала** после N измерений (для унимодальной функции).

Числа Фибоначчи. $F_0 = F_1 = 1$, далее $F_{m+1} = F_m + F_{m-1}$.

Идея размещения точек. На первом шаге на отрезке длины L ставят две точки на расстояниях $F_{N-1}/F_N \cdot L$ и $F_{N-2}/F_N \cdot L$ от одного конца (формулы в разных учебниках записывают через отношения F_{N-k}/F_N). После сравнения f интервал сокращается, а **одна** из точек остаётся внутри нового интервала в нужном отношении для оставшегося числа измерений $N - 1$.

Связь с золотым сечением. При $N \rightarrow \infty$ отношения F_{N-1}/F_N стремятся к τ , поэтому для “очень большого числа шагов” метод Фибоначчи близок к золотому сечению, но на конечном N он *оптимален* по длине финального интервала.

3.6 Метод Свенна (расширение интервала)

Зачем: когда нет готового $[a, b]$, но есть стартовая точка x_0 “где-то рядом” с долиной минимума.

Идея: идти из x_0 в сторону убывания f , увеличивая шаг (например, геометрически: $h, 2h, 4h, \dots$), пока не получится тройка точек

$$x_{k-1} < x_k < x_{k+1}, \quad f(x_{k-1}) \geq f(x_k) \leq f(x_{k+1}),$$

обрамляющая минимум. После этого к обрамляющему отрезку применяют золотое сечение.

3.7 Параболическая интерполяция (квадратичная аппроксимация)

Идея: по трём точкам с разными значениями f построить параболу $q(x)$, совпадающую с f в этих точках, и перейти к её минимуму. Затем обновить тройку точек, сохраняя обрамление.

Когда осторожно: если f шумная или тройка плохая, парабола может вести куда угодно; в библиотеках обычно комбинируют с золотым сечением (метод Брента).

4 Многомерная безусловная оптимизация: методы спуска в $\mathbb{R}^n$

Общая схема итерации:

$$x_{k+1} = x_k + \alpha_k p_k,$$

где p_k — направление (часто “вниз” по рельефу f), $\alpha_k > 0$ — длина шага.

4.1 Направление спуска

Вектор p_k называют **направлением спуска** в точке x_k , если

$$\nabla f(x_k)^\top p_k < 0.$$

Тогда при достаточно малом $\alpha > 0$ выполняется $f(x_k + \alpha p_k) < f(x_k)$ (из разложения Тейлора первого порядка).

4.2 Линейный поиск (подбор шага α)

Зачем. Имеем направление p_k (например, $-\nabla f(x_k)$ или квазиньютоновское). Хотим $\alpha > 0$ такое, что $f(x_k + \alpha p_k)$ существенно меньше $f(x_k)$, но без “прыжка” через долину, где линейная модель уже лжёт.

Модель первого порядка. Локально $f(x_k + \alpha p_k) \approx f(x_k) + \alpha \nabla f(x_k)^\top p_k$. Если p_k — направление спуска, то $\nabla f(x_k)^\top p_k < 0$, и при малых α значение убывает линейно по α .

Условие Армико (достаточное убывание): для $c_1 \in (0, 1)$ (часто 10^{-4}) ищут $\alpha > 0$:

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^\top p_k.$$

Правая часть — это прямая “ниже” линейной модели; требование: реальное f должно оказаться под этой прямой.

Backtracking (затухающий шаг). Берут $\alpha = \alpha_0$ (например, 1), пока не выполнится Армико, делают $\alpha \leftarrow \rho \alpha$ с $\rho \in (0, 1)$ (например, 0,5). Это дёшево: несколько вычислений f .

Сильное правило Вольфе (кривизна). Часто добавляют второе условие с $c_2 \in (c_1, 1)$ (часто 0,9):

$$|\nabla f(x_k + \alpha p_k)^\top p_k| \leq c_2 |\nabla f(x_k)^\top p_k|.$$

Упрощённый вариант (слабое Вольфе): $\nabla f(x_k + \alpha p_k)^\top p_k \geq c_2 \nabla f(x_k)^\top p_k$.

Смысл. Если производная $\frac{d}{d\alpha} f(x_k + \alpha p_k) = \nabla f(x_k + \alpha p_k)^\top p_k$ слишком отрицательна в конце шага, значит *вдоль* p_k ещё есть крутой спуск — шаг можно было продолжить, но часто это признак, что мы уже близко к “дну” и квазиньютоновская модель требует обновления; Вольфе балансирует длину шага.

4.3 Градиентный спуск

Зачем: базовый метод для гладких функций, когда есть градиент.

Идея “с нуля”: в точке x_k функцию *подменяют* самой простой нелинейной моделью — **линейной** (это и есть разложение Тейлора **первого** порядка):

$$f(x_k + s) \approx f(x_k) + \nabla f(x_k)^\top s.$$

Если мы хотим *сразу* сильно уменьшить эту линейную модель, логично идти в сторону, противоположную градиенту: $s = -\nabla f(x_k)$. Но линейная модель при уходе далеко от

x_k перестаёт походить на настоящую f , поэтому делают **короткий** шаг α_k и повторяют заново в новой точке.

Формулы: $x_{k+1} = x_k - \alpha_k \nabla f(x_k)$.

Про постоянный шаг: если ∇f липшицев с константой L (часто так для гладких функций с ограниченной второй производной), то при $\alpha \in (0, 2/L)$ метод с постоянным α монотонно уменьшает f (в выпуклом случае — классическая оценка).

Проблемы: в “узких долинах” траектории сильно осциллируют; сходимость может быть медленной.

Как выбрать α на практике. Константный шаг (если известна L -гладкость), либо Армихо/backtracking на каждом шаге. Для выпуклых задач иногда используют Barzilai–Borwein или адаптивные правила — это уже улучшения базовой схемы.

4.4 Метод наискорейшего спуска

Идея. Направление то же: $p_k = -\nabla f(x_k)$, но вместо “угадывания” α_k решают одномерную задачу

$$\alpha_k = \arg \min_{\alpha \geq 0} \varphi(\alpha), \quad \varphi(\alpha) := f(x_k - \alpha \nabla f(x_k)).$$

Функция $\varphi(\alpha)$ — это f на луче из x_k вдоль антиградиента. Её минимум — **точка касания** уровня f лучом (в гладком выпуклом случае).

Как искать α_k . Одномерный метод (золотое сечение, параболы + Брент) или приближённый поиск с Вольфе.

Плюсы. За один “большой” шаг можно сильно уменьшить f , если овраг не слишком кривой.

Минусы. Каждый шаг дороже; для плохо обусловленных квадратичных функций траектории всё равно могут зигзагообразно идти к минимуму — это не лечится точным α_k вдоль градиента.

4.5 Метод сопряжённых градиентов (для квадратичной функции)

Рассмотрим $f(x) = \frac{1}{2}x^\top Ax - b^\top x$ с симметричной положительно определённой матрицей $A \succ 0$.

Идея “с нуля”. Градиент в точке x : $g(x) = Ax - b$. Минимум — решение $Ax = b$. Можно было бы решать СЛАУ напрямую, но если A большая и разреженная, полезны итерационные методы. CG строит направления $p_0, p_1, \dots$, которые **A -ортогональны** (сопряжены): $p_i^\top A p_j = 0$ при $i \neq j$. Тогда каждый шаг уменьшает ошибку в “новом” направлении, не портя уже достигнутое вдоль предыдущих; для квадратичной функции за $\leq n$ шагов (точная арифметика) получается точное решение.

Величины. Невязка $r_k := g(x_k) = Ax_k - b$. Начало: $p_0 = -r_0$.

Итерация (Флетчер–Ривс):

$$r_k = Ax_k - b, \quad p_0 = -r_0,$$

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k,$$

$$r_{k+1} = r_k + \alpha_k A p_k, \quad \beta_{k+1} = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}, \quad p_{k+1} = -r_{k+1} + \beta_{k+1} p_k.$$

Замечание про β_k . В формуле Флетчера–Ривса $\beta_{k+1} = \|r_{k+1}\|^2 / \|r_k\|^2$. Другой популярный вариант — Полак–Рибьер:

$$\beta_{k+1}^{\text{PR}} = \frac{r_{k+1}^\top (r_{k+1} - r_k)}{\|r_k\|^2}.$$

Для **нелинейного** $f(x)$ направления перестают быть точно A -сопряжёнными; тогда на каждом шаге берут p_k по той же схеме, но $r_k = \nabla f(x_k)$, а α_k находят линейным поиском (часто Вольфе), иногда с дополнительным условием, чтобы p_k оставался спуском.

4.6 Метод Ньютона (подробно для начинающих)

Что мы вообще хотим. В безусловной минимизации ищут точку, где градиент обнуляется: $\nabla f(x^*) = 0$. Это **не то же самое**, что “найти корень произвольной функции”, но очень похоже: если ввести $g(x) = \nabla f(x)$, то для минимума нужно решить уравнение $g(x) = 0$. Классический метод Ньютона–Рафсона для уравнения использует производную; в оптимизации на роль “производной” выступает **матрица вторых производных** — Гессиан.

4.6.1 Одна переменная: от ряда Тейлора до формулы “минус первая вторая”

Зафиксируем текущую точку x_k и обозначим малое смещение t (шаг вдоль оси x). Разложим f в ряд Тейлора **до второго порядка** около x_k :

$$f(x_k + t) \approx q_k(t) := f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2.$$

Геометрически $q_k(t)$ — это **парабола** по t . Если $f''(x_k) > 0$, парабола “улыбкой вверх” и у неё есть единственный минимум.

Чтобы найти минимум $q_k(t)$, возьмём производную по t и приравняем нулю:

$$\frac{d}{dt}q_k(t) = f'(x_k) + f''(x_k)t = 0 \quad \Rightarrow \quad t_k = -\frac{f'(x_k)}{f''(x_k)}.$$

Важная мелочь про “первую делить на вторую”: в учебниках часто говорят коротко “шаг пропорционален производным”, но знак уже в формуле: это именно $-f'/f''$, а не $+f'/f''$. Минус означает: если $f'(x_k) > 0$ (функция локально растёт), нужно сдвинуться *влево*, если $f' < 0$ — *вправо*.

Ньютоновская итерация в одномерии записывается так:

$$x_{k+1} = x_k + t_k = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Мини-пример. Пусть $f(x) = \frac{1}{2}ax^2 + bx + c$ при $a > 0$. Тогда $f'(x) = ax + b$, $f''(x) = a$. Один шаг Ньютона из любой точки x_k даёт

$$x_{k+1} = x_k - \frac{ax_k + b}{a} = -\frac{b}{a},$$

то есть **сразу** точный минимум параболы. Для общей гладкой функции парабола лишь *приближает* f около x_k , поэтому шаги повторяют.

4.6.2 Много переменных: то же самое, но “вторая производная” — матрица

Пусть $f : \mathbb{R}^n \rightarrow \mathbb{R}$ дважды дифференцируема. Вектор неизвестного шага обозначим $s \in \mathbb{R}^n$. Разложение Тейлора второго порядка около x_k :

$$f(x_k + s) \approx m_k(s) := f(x_k) + \nabla f(x_k)^\top s + \frac{1}{2} s^\top H(x_k) s,$$

где $H(x_k) = \nabla^2 f(x_k)$ — симметричная матрица (Гессиян) из вторых частных производных.

Мы снова минимизируем **квадратичную модель** $m_k(s)$. У квадратичной функции на $\mathbb{R}^n$ градиент по s линейен; условие $\nabla_s m_k(s) = 0$ даёт

$$\nabla f(x_k) + H(x_k)s = 0 \quad \Leftrightarrow \quad H(x_k)s = -\nabla f(x_k).$$

Если $H(x_k)$ обратима, то формально $s_k = -H(x_k)^{-1} \nabla f(x_k)$ — это прямое обобщение формулы $t = -f'/f''$: вместо одной второй производной стоит **обратная матрица** кривизн.

Обновление точки:

$$x_{k+1} = x_k + s_k.$$

4.6.3 Как это считают на практике (и почему не “инвертируют Гессиян”)

Обычно **не вычисляют** H^{-1} явно: это дорого. Решают систему линейных уравнений

$$H(x_k)s_k = -\nabla f(x_k)$$

методом Холецкого/LU/итерационно — получают только вектор s_k .

4.6.4 Когда шаг Ньютона “хороший”, а когда опасный

- Если $H(x_k) \succ 0$ (все собственные значения > 0), то s_k — направление **спуска** (для малых шагов f убывает вдоль s_k), и квадратичная модель действительно имеет минимум.
- Если $H(x_k)$ не знакопостоянна (есть отрицательные собственные значения), “минимум модели” может быть седловым или не тем, что нужно: направление может увести к **максимуму** или седлу исходной f .
- Если H близка к вырожденной, решение СЛАУ неустойчиво.

Поэтому в коде почти всегда стоят **демпфирование** (шаг $x_{k+1} = x_k + \alpha_k s_k$ с подбором α_k) и/или **сдвиг** матрицы $H + \lambda I$, и/или доверительные области (trust region).

Плюсы: около решения, где модель близка к правде и $H \succ 0$, сходимость очень быстрая (“квадратичная” по ошибке — на каждом шаге число верных цифр растёт примерно вдвое).

Минусы: цена шага $O(n^3)$ при плотном H наивным решением СЛАУ, плюс нужно получать вторые производные (аналитически, автодифференцированием или конечными разностями).

Удобное вводное чтение (в т. ч. вывод $t = -f'(x_k)/f''(x_k)$ и переход к $\mathbb{R}^n$): [Wikipedia: Newton’s method in optimization](#).

4.7 Модификации Ньютона (что именно улучшают)

- **Дампированный Ньютон:** $x_{k+1} = x_k + \alpha_k s_k$, где α_k подбирают по Армихо/Вольфе.
- **Сдвиг (Levenberg–Marquardt идея):** $(H(x_k) + \lambda_k I)s_k = -\nabla f(x_k)$, чтобы матрица была лучше обусловлена.
- **Усечённый Ньютон:** решают $Hz = -g$ приближённо итерациями, не формируя H явно (матвекторы).

4.8 Квазиньютоновские методы (DFP, BFGS) — полная запись

Зачем. Метод Ньютона требует на каждом шаге точный Гессиян $\nabla^2 f(x_k)$ и решение СЛАУ. Квазиньютоновские методы хранят **приближение** кривизны и обновляют его дёшево (поправка ранга 1–2), используя только градиенты в соседних точках.

Обозначения.

- $s_k := x_{k+1} - x_k$ — фактический сдвиг на шаге;
- $y_k := \nabla f(x_{k+1}) - \nabla f(x_k)$ — изменение градиента;
- B_k — симметричная аппроксимация **Гессияна**; $H_k := B_k^{-1}$ — аппроксимация **обратного** Гессияна (в коде BFGS часто обновляют именно H_k).

Условие секущей. Требуют, чтобы новая матрица согласовалась с наблюдением “ $y \approx Bs$ ” на последнем шаге:

$$B_{k+1}s_k = y_k.$$

Для квадратичной f с матрицей Q равенство $y = Qs$ выполнялось бы точно; для общей функции мы подгоняем только последний шаг.

Почему нужно $y_k^\top s_k > 0$. Для выпуклой квадратичной функции это выполняется строго. При линейном поиске по правилам **Вольфе** для минимизации это неравенство обычно сохраняется — тогда обновления DFP/BFGS сохраняют положительную определённость B_k (при $B_0 \succ 0$).

4.8.1 Каркас алгоритма квазиньютоновского спуска

1. Задать x_0 , симметричную $B_0 \succ 0$ (часто $B_0 = I$).
2. Решить $B_k p_k = -\nabla f(x_k)$ (или $p_k = -H_k \nabla f(x_k)$, если храните H_k).
3. Линейный поиск: найти α_k , положить $s_k = \alpha_k p_k$, $x_{k+1} = x_k + s_k$.
4. $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. Если $y_k^\top s_k \leq 0$, не обновлять матрицу или уменьшать шаг.
5. Обновить B_k или H_k по DFP/BFGS.

4.8.2 DFP (обновление аппроксимации Гессияна B_k)

Пусть $y_k^\top s_k > 0$ и $s_k^\top B_k s_k > 0$. Стандартная форма:

$$B_{k+1} = B_k + \frac{y_k y_k^\top}{y_k^\top s_k} - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k}.$$

4.8.3 BFGS (обновление B_k)

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}.$$

4.8.4 BFGS (обновление $H_k \approx B_k^{-1}$, удобно в реализации)

Положим $\rho_k := 1/(y_k^\top s_k)$. Тогда

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top.$$

Направление $p_{k+1} = -H_{k+1} \nabla f(x_{k+1})$ получается без решения СЛАУ (стоимость $O(n^2)$ при плотной H_k).

4.8.5 Сложность и L-BFGS

Плотное хранение B_k или H_k : память $O(n^2)$. Для больших n используют **L-BFGS**: хранят только $m \ll n$ последних пар (s_j, y_j) и восстанавливают действие $H_k g$ за $O(mn)$.

Справочно: [Wikipedia: BFGS](#).

5 Методы без производных и покоординатный спуск

5.1 Метод Гаусса–Зейделя (покоординатный спуск) — как читать формулу

Зачем. Свести сложную задачу в $\mathbb{R}^n$ к серии простых одномерных минимизаций.

В чём путаница. В короткой записи

$$x^{(i)} \leftarrow \arg \min_{\alpha} f(x^{(1)}, \dots, x^{(i-1)}, \alpha, x^{(i+1)}, \dots, x^{(n)})$$

не видно, какие $x^{(j)}$ уже **новые**, а какие ещё **старые**. В методе **Гаусса–Зейделя** при минимизации по i -й координате используют уже обновлённые координаты $1, \dots, i-1$ и ещё не тронутые $i+1, \dots, n$.

Ясная запись через промежуточные векторы $z^{(i)}$. Пусть $x^{[t]}$ — вектор перед очередным полным проходом. Положим $z^{(0)} = x^{[t]}$. Для каждого $i = 1, \dots, n$ определим $z^{(i)} \in \mathbb{R}^n$ так:

$$z_j^{(i)} = \begin{cases} z_j^{(i-1)}, & j \neq i, \\ \arg \min_{\alpha \in \mathbb{R}} f(z_1^{(i-1)}, \dots, z_{i-1}^{(i-1)}, \alpha, z_{i+1}^{(i-1)}, \dots, z_n^{(i-1)}), & j = i. \end{cases}$$

После цикла $i = 1, \dots, n$ кладут $x^{[t+1]} := z^{(n)}$.

Отличие от метода Якоби. В методе Якоби все координаты на одном проходе считают из *одного* $x^{[t]}$, затем разом делают $x^{[t+1]}$. У Гаусса–Зейделя информация “протекает” по координатам сразу — часто быстрее, но сложнее распараллелить.

Остановка. $\|x^{[t+1]} - x^{[t]}\| \leq \varepsilon$ или малость $|f(x^{[t+1]}) - f(x^{[t]})|$.

Когда хорош. Частично разделимые f , дешёвые одномерные подзадачи. **Когда плох.** Сильные “овраги” — как у градиентного спуска, много проходов без большого эффекта.

5.2 Метод Хука–Дживса (pattern search)

Зачем. Минимизировать $f : \mathbb{R}^n \rightarrow \mathbb{R}$ без градиента; допустимы шум и “чёрный ящик”. Хорош при n небольшом (десятки).

Идея. Две фазы чередуются:

- **Разведка (exploratory move):** из текущей базовой точки пробуют маленькие смещения $\pm\Delta$ по каждой оси по очереди. Если по оси i нашлось улучшение, i -я координата остаётся лучшей; если нет — возвращают старое значение.
- **Шаг по образцу (pattern move):** если разведка улучшила точку, строят направление “куда мы вообще ушли за разведку” $d = x^{\text{new}} - x^{\text{base}}$ и пробуют точку

$$x^{\text{pat}} = x^{\text{new}} + \gamma d, \quad \gamma > 0 \text{ (часто } \gamma = 1 \text{)}.$$

Вокруг x^{pat} снова делают разведку. Идея: если долина вытянута, суммарный сдвиг по осям указывает вдоль долины, и шаг по образцу ускоряет движение.

Параметры. Стартовый Δ , минимальный $\Delta_{\min}$, коэффициент γ , правило деления Δ (часто $\Delta \leftarrow \Delta/2$), если разведка из базовой точки *не* дала улучшения.

Псевдокод одной “большой” итерации.

1. Задать базовую точку x_b .
2. **Разведка:** $x \leftarrow x_b$; для $i = 1, \dots, n$: если $f(x - \Delta e_i) < f(x)$, то $x \leftarrow x - \Delta e_i$; иначе если $f(x + \Delta e_i) < f(x)$, то $x \leftarrow x + \Delta e_i$.
3. Если $f(x) < f(x_b)$: положить $d = x - x_b$, $x_{\text{trial}} = x + \gamma d$; выполнить разведку от x_{trial} ; если улучшение есть, принять новую базу, иначе базой остаётся x .
4. Если на шаге 2 не было улучшения относительно x_b , уменьшить Δ .

Остановка. $\Delta \leq \Delta_{\min}$ или исчерпан бюджет вызовов f .

Плюсы/минусы. Плюс: простота, нет производных. Минус: при больших n и оврагах может быть очень медленным; нет глобальной сходимости в общем случае.

5.3 Метод Пауэлла (метод сопряжённых направлений без градиента)

Зачем. Улучшить покоординатный спуск: в “овраге” последовательные минимизации по осям дают зигзаги; Пауэлл добавляет направление “куда мы реально продвинулись за весь цикл”.

Данные. Начальная точка x_0 , линейно независимые направления $d_1, \dots, d_n$ (обычно $d_i = e_i$).

Один цикл (основная схема).

1. Положить $z_0 := x$ (текущая лучшая точка).
2. Для $i = 1, \dots, n$: найти $\alpha_i \in \mathbb{R}$, минимизирующее $\varphi_i(\alpha) = f(z_{i-1} + \alpha d_i)$ (одномерный поиск), и положить

$$z_i := z_{i-1} + \alpha_i d_i.$$

3. Суммарный сдвиг за цикл:

$$s := z_n - z_0 = \sum_{i=1}^n \alpha_i d_i.$$

4. **Обновление направлений (классическая идея Пауэлла).** Удалить одно из старых направлений (часто то, вдоль которого было *наименьшее* уменьшение f — интуиция: оно “менее полезно”), а на его место поставить s (после нормализации или без — в разных изложениях по-разному). Остальные направления сдвинуть по порядку, чтобы снова было ровно n направлений.

5. Положить $x \leftarrow z_n$ и перейти к следующему циклу.

Важное замечание. В оригинальном методе Пауэлла нужна проверка, чтобы направления не стали почти линейно зависимыми (иначе одномерные поиски “ломаются”). В учебных конспектах это часто опускают, но в реализации добавляют критерий или перезапускают направления на оси.

Остановка. $\|s\| \leq \varepsilon$ или малость прироста f .

5.4 Нелдер–Мид: симплекс в $\mathbb{R}^n$

Зачем. То же, что Хук–Дживс: нет градиента, f — чёрный ящик; размерность обычно небольшая.

Геометрия. В $\mathbb{R}^n$ берут $n + 1$ точку $x_1, \dots, x_{n+1}$ в общем положении (не лежащие в одной гиперплоскости). Их выпуклая оболочка — симплекс (сегмент при $n = 1$, треугольник при $n = 2, \dots$).

Обозначения на итерации. Пусть вершины пронумерованы так, что

$$f(x_1) \leq f(x_2) \leq \dots \leq f(x_{n+1}),$$

то есть x_{n+1} — худшая, x_1 — лучшая. Центр “хорошей” грани (без худшей вершины):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Отражение (reflection). Параметр $\rho > 0$ (часто 1):

$$x_r = \bar{x} + \rho(\bar{x} - x_{n+1}).$$

Если $f(x_r)$ лучше, чем у второй по плохости, но не лучше $f(x_1)$, заменяют $x_{n+1} \leftarrow x_r$.

Растяжение (expansion). Если $f(x_r) < f(x_1)$ (отражение дало новый рекорд), пробуют

$$x_e = \bar{x} + \chi(x_r - \bar{x}), \quad \chi > 1 \text{ (часто 2)}.$$

Если $f(x_e)$ ещё лучше, $x_{n+1} \leftarrow x_e$, иначе $x_{n+1} \leftarrow x_r$.

Сжатие внешнее (outside contraction). Если x_r не лучше второй худшей, но лучше старой худшей, иногда делают

$$x_c = \bar{x} + \gamma(x_r - \bar{x}), \quad 0 < \gamma < 1.$$

Сжатие внутреннее (inside contraction). Если x_r хуже всех кроме старой худшей, сжимают “внутрь”:

$$x_{cc} = \bar{x} - \gamma(\bar{x} - x_{n+1}).$$

Глобальное уменьшение (shrink). Если сжатия не помогли, все вершины кроме лучшей стягивают к x_1 :

$$x_i \leftarrow x_1 + \sigma(x_i - x_1), \quad i = 2, \dots, n+1, \quad 0 < \sigma < 1 \text{ (часто 0,5)}.$$

Остановка. Малость симплекса $\max_i \|x_i - x_1\| \leq \varepsilon$ или малость разброса значений f .

Плюсы/минусы. Плюс: очень простая логика, мало кода. Минус: нет общих гарантий сходимости; при большом n обычно плох; при “плоской” f симплекс вырождается.

6 Условная оптимизация: штрафы и барьеры

6.1 Метод штрафных функций

Исходная задача. $\min_{x \in \Omega} f(x)$, где $\Omega = \{x : g_i(x) \leq 0, h_j(x) = 0\}$.

Идея “штрафа”. Вместо жёстких ограничений вводят гладкую функцию $P(x) \geq 0$, которая равна нулю на Ω и быстро растёт при выходе наружу. Решают последовательность задач

$$\min_{x \in \mathbb{R}^n} F_\rho(x) = f(x) + \rho P(x), \quad \rho > 0.$$

При $\rho \rightarrow +\infty$ минимизаторы F_ρ обычно стремятся к решению исходной задачи (при стандартных предположениях).

Пример квадратичного штрафа для неравенств и равенств.

$$P(x) = \sum_{i=1}^m (\max\{0, g_i(x)\})^2 + \sum_{j=1}^p (h_j(x))^2.$$

Тогда вне Ω добавка ρP “штрафует” за нарушение: чем дальше за границу, тем больше P .

Градиент (если нужен для BFGS). Для одного слагаемого $(\max\{0, g\})^2$ в точках, где $g(x) < 0$, производная ноль; где $g(x) > 0$, $\nabla = 2g(x)\nabla g(x)$. На границе $g = 0$ гладкость теряется — на практике работают почти всегда вне “осколков”.

Внешний цикл (как программируют).

1. Выбрать ρ_1 (не слишком большое), старт $x^{(0)}$.
2. Для $s = 1, 2, \dots$: решить $\min_x F_{\rho_s}(x)$ приближённо, взяв старт с решения при ρ_{s-1} ; увеличить $\rho_{s+1} = \sigma \rho_s$ ($\sigma > 1$).
3. Остановиться, когда $\max_i g_i(x) \leq \varepsilon_g$ и $|h_j(x)| \leq \varepsilon_h$.

Минусы. При больших ρ задача становится **плохо обусловленной** (узкие овраги вдоль границы), градиенты огромны — нужны устойчивые оптимизаторы и иногда регуляризация.

6.2 Метод барьерных функций (внутренние точки)

Идея. Для строгих неравенств $g_i(x) < 0$ строят вспомогательную задачу *только для внутренних* точек:

$$\min_{x \in \text{int } \Omega} B_\mu(x), \quad B_\mu(x) := f(x) - \mu \sum_{i=1}^m \ln(-g_i(x)), \quad \mu > 0.$$

Логарифм $\ln(-g_i)$ стремится к $+\infty$, когда $g_i \rightarrow 0^-$ — как “невидимая стена” у границы.

Поведение при $\mu \rightarrow 0$. Чем меньше μ , тем слабее барьер далеко от границы, и минимум B_μ ближе к решению исходной задачи (часто к точке на границе).

Градиент барьерной задачи.

$$\nabla B_\mu(x) = \nabla f(x) - \mu \sum_{i=1}^m \frac{\nabla g_i(x)}{g_i(x)}.$$

У границы члены $\mu/|g_i|$ велики — шаги делают малыми.

Связь с внутренне-точечными методами. Логарифмический барьер — ключевая идея primal-dual interior-point методов для ЛП и выпуклой оптимизации.

7 Линейное программирование (ЛП)

Задача. Линейная целевая функция и линейные ограничения; стандартная форма для интуиции:

$$\min_{x \in \mathbb{R}^n} c^\top x \quad \text{при} \quad Ax \leq b, \quad x \geq 0.$$

Здесь $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

Геометрия. Множество $\{x : Ax \leq b, x \geq 0\}$ — выпуклый **многогранник** (пересечение конечного числа полупространств). Линейная функция на многограннике достигает экстремума в вершине (если многогранник ограничен в направлении минимизации).

Симплекс-метод (словами). Стартуют из допустимой вершины. На каждом шаге выбирают **входящую** переменную (по отрицательной “оценке” в строке цели) и **выходящую** (минимальное положительное отношение правых частей к положительным коэффициентам — правило минимального отношения), чтобы перейти по ребру в соседнюю вершину с не большей целевой функцией. Пока есть улучшение — продолжают.

Табличная форма. На зачёте часто просят уметь читать симплекс-таблицу: базисные переменные, приведённая целевая строка, допустимость.

Внутренне-точечные методы. Идут через *внутренность* допустимого множества (барьеры/примал-дуальные шаги), часто полиномиальная сложность в теории; на практике конкурируют с симплексом на больших разреженных задачах.

Справочно: [Wikipedia: Simplex algorithm](#).

8 Глобальная оптимизация: PSO и генетические алгоритмы

Это **эвристики**: глобальный минимум не гарантирован, но на практике часто находят хорошее решение при нескольких запусках и настройке параметров.

8.1 Рой частиц (PSO, Particle Swarm Optimization)

Биологическая метафора. Стая “летит” по пространству параметров; каждая частица помнит свой лучший найденный пункт, а также знает (или знает у соседей) лучшую точку роя.

Состояние. Частица с номером i : позиция $x_i \in \mathbb{R}^n$, скорость $v_i \in \mathbb{R}^n$. Индивидуальный лучший минимум p_i (personal best), глобальный лучший g (global best). В варианте **lbest** вместо g используют лучшую точку среди соседей по заданной топологии.

Классическое обновление (инерционная модель). На шаге t :

$$\begin{aligned} v_i^{t+1} &= \omega v_i^t + c_1 r_1^t \odot (p_i - x_i^t) + c_2 r_2^t \odot (g - x_i^t), \\ x_i^{t+1} &= x_i^t + v_i^{t+1}, \end{aligned}$$

где r_1^t, r_2^t — векторы из $U(0, 1)$ по координатам (поэлементное умножение $\odot$), $\omega, c_1, c_2 > 0$.

Смысл коэффициентов. ω — инерция (больше ω — больше “разлёт” и исследование). c_1 — вес “к своей памяти”; c_2 — вес “к общему лидеру”.

Ограничения. Часто вводят $v_i \leftarrow \text{clip}(v_i, -v_{\max}, v_{\max})$ и x_i режут по допустимой области (проекция или штраф).

Псевдокод.

1. Инициализировать x_i, v_i , вычислить $f(x_i)$, задать $p_i = x_i$, найти g .

2. Повторять: обновить v_i, x_i ; если $f(x_i) < f(p_i)$, то $p_i \leftarrow x_i$; если $f(x_i) < f(g)$, то $g \leftarrow x_i$; критерий останова.

8.2 Генетический алгоритм (ГА)

Кодирование. Бинарное: хромосома — строка битов; **вещественное:** хромосома — вектор $x \in \mathbb{R}^n$.

Fitness. Для минимизации f задают, например, $\text{fitness}(x) = -f(x)$ (чем больше fitness, тем лучше).

Турнирная селекция размера T . Для выбора одного родителя: взять T случайных особей, выбрать лучшую по fitness. Повторить, чтобы набрать пару родителей.

Кроссинговер (примеры).

- **Одноточечный (бинарный):** разрезать две строки в позиции k и обменять хвосты.
- **Арифметический (вещественный):** для родителей $a, b \in \mathbb{R}^n$ и случайного $\lambda \in [0, 1]$ потомок $c = \lambda a + (1 - \lambda)b$ (иногда два потомка симметрично).

Мутация. Бинарная: инвертировать каждый бит с вероятностью p_m . Вещественная: $x \leftarrow x + \sigma \xi$, $\xi \sim \mathcal{N}(0, I)$.

Элитизм. Сохранить E лучших особей из поколения t в поколение $t+1$ без изменений — чтобы не потерять лучшее решение.

Схема поколения.

1. Инициализация популяции; оценка fitness.
2. Пока не стоп: селекция $\rightarrow$ кроссинговер $\rightarrow$ мутация $\rightarrow$ формирование нового поколения с элитизмом.

Баланс. Сильная селекция и слабая мутация — быстрая сходимость к локальному оптимуму; слабая селекция и сильная мутация — долгое блуждание.

9 Имитация отжига (SA): от порога к вероятности

Задача. $\min_{X \in \Omega} \varphi(X)$, где Ω — дискретное или непрерывное множество допустимых решений (перестановки, целочисленные векторы, вещественный ящик и т.д.).

Почему не жадный локальный поиск. Если на каждом шаге брать только улучшения $\varphi(X') < \varphi(X)$, легко застрять в локальном минимуме. Отжиг разрешает иногда **ухудшения**, чтобы “выпрыгнуть” из ямы.

Окрестность. Для каждого $X \in \Omega$ задано конечное (или измеримое) множество соседей $d(X) \subseteq \Omega$. На шаге из X_t выбирают случайного соседа $X'_t \in d(X_t)$ (равномерно или по заданным весам).

9.1 Пороговый алгоритм (мотивация)

Детерминированная эвристика: принимать X' , если $\varphi(X') - \varphi(X) < \varepsilon(t)$, где $\varepsilon(t) \rightarrow 0$. Тогда в начале допускаются большие ухудшения, в конце — почти жадность. Недостаток: неясно, как выбрать $\varepsilon(t)$ универсально.

9.2 Правило Метрополиса (стандарт SA для минимизации)

Положим $\Delta_t = \varphi(X'_t) - \varphi(X_t)$ — приращение цели (хотим минимум, значит $\Delta_t < 0$ — улучшение). Вероятность принятия кандидата:

$$p_{\text{acc}}(\Delta_t, T_t) = \begin{cases} 1, & \Delta_t \leq 0, \\ \exp(-\Delta_t/T_t), & \Delta_t > 0, \end{cases}$$

где $T_t > 0$ — температура. Генерируют $u_t \sim U(0, 1)$; если $u_t < p_{\text{acc}}$, то $X_{t+1} = X'_t$, иначе $X_{t+1} = X_t$.

Связь с Больцманом (для интуиции). В статфизике вероятности состояний пропорциональны $\exp(-E/kT)$; здесь φ играет роль энергии, T — “шум”.

Поведение при разных T . При большом T_t даже $\Delta_t \gg 0$ даёт $\exp(-\Delta/T) \approx 1$ — почти всё принимается (исследование). При малом T_t ухудшения почти не проходят — локальная доводка.

9.3 Полный псевдокод (с лучшим найденным решением)

1. Выбрать $X \in \Omega$, вычислить $\varphi(X)$; положить $X_{\text{best}} \leftarrow X$, $\varphi_{\text{best}} \leftarrow \varphi(X)$.
2. Задать $T \leftarrow T_0$, функцию охлаждения $T \leftarrow g(T, t)$, параметры остановки.
3. Для $t = 1, 2, \dots$: сгенерировать $X' \in d(X)$, $\Delta \leftarrow \varphi(X') - \varphi(X)$; принять по правилу Метрополиса; если $\varphi(X) < \varphi_{\text{best}}$, обновить рекорд; обновить T ; проверить стоп.

9.4 Законы охлаждения (как в методичке)

- **Геометрическое:** $T_t = \alpha T_{t-1}$, $\alpha \in [0,8; 0,99]$ — простой и частый выбор на практике.
- **Логарифмическое:** $T_t = T_0 / \ln(t + 1)$ — теоретически связано с медленным охлаждением (на практике может быть очень медленным).
- **Very fast annealing:** $T_t = T_0 \exp(-\beta t^{1/|X|})$, где $|X|$ трактуют как размерность решения.

9.5 Практические советы

- **Окрестность** должна быть достаточно “богатой”: из любого разумного состояния за несколько шагов можно добраться до области глобально хороших решений (на графе состояний — связность).
- **Начальная температура** часто подбирают так, чтобы в начале принималась заметная доля ухудшающих шагов (например, 20–60% для умеренных Δ).
- **Остановка:** $T_t < T_{\min}$, лимит итераций, либо нет улучшения рекорда K шагов подряд.

10 Краткая памятка: что выбрать

- **Одна переменная, унимодальность, отрезок известен:** золотое сечение или Фибоначчи.

- **Отрезок неизвестен:** Свенн + золотое сечение.
- **Гладкая f , есть ∇f :** градиентный/наискорейший; для скорости и устойчивости — BFGS с Вольфе.
- **Квадратичная положительно определённая A :** сопряжённые градиенты.
- **Нет производных, мало размерность:** Нелдер–Мид или Хук–Дживс.
- **Ограничения “в лоб”:** штрафы/барьеры + внутренний метод.
- **Много локальных минимумов / комбинаторика:** SA, ГА, PSO; часто + локальная доводка.